

# Development of autonomous robots with BDI agents and ROS

**Maiquel de Brito**

<sup>1</sup>UFSC-Blumenau-Brasil  
maiquel.b@ufsc.br

# Preliminaries

## **What are we going to see in this course?**

- Programming ROS-based robots controlled by Jason agents
- Essentials on Jason and ROS

## **What are we not going to see in this course?**

- Details on ROS
- Details on Jason

Practical activities are available [here](#)

# — Part I —

## ROS

# What is ROS?

## **ROS — Robot Operating System:**

A flexible framework for writing robot software

Provides tools and libraries to help developers to create robot applications

### **Key Components:**

- **Middleware:** For inter-process communication and hardware abstraction
- **Tools:** For development, debugging, and visualization.
- **Ecosystem:** Large community and a vast repository of packages.

# The Turtlesim simulation

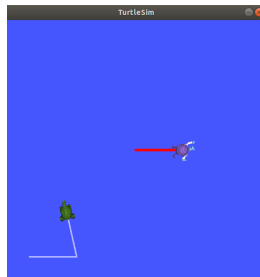
The hands on part of this course uses the well-known [turtlesim](#) simulator with some extensions

One or more turtle-shaped robots in a square world

whose sensors and actuators are controlled by ROS

The robots can freely move and leave a trail wherever they go

Positions in the environment follow the cartesian coordinate system being thus given by the coordinates  $(x, y)$



Go to the Preliminaries – Running ROS section of the [tutorial](#)

# ROS Architecture

**Nodes:** Processes that perform computation.

**Topics:** Named buses for asynchronous communication.

**Services:** Synchronous remote procedure calls.

**Messages:** Data structures for communication.

[ROS architecture scheme](#)

# ROS Nodes

Processes that perform computation.

the smallest executable unit of ROS – either Python scripts or C++ compiled files

Responsible for a single, modular purpose

e.g. one node for controlling wheel motors, one node for controlling a laser range-finder, etc.

Can interact with each other even running in different machines

following some interaction pattern:

- publish/subscribe (topics)
- client/server (services)
- action client/action server (actions)

Launching a node: `ros2 run <package name> <node name>`

more on ROS nodes [here](#)

# ROS Topics

Topics are data containers

Nodes may

- publish data to any number of topics  
multiple nodes can publish data to the same topic
- subscribe to any number of topics  
subscribers are updated when the topic data changes

[ROS topic illustration](#)

## useful command line topic tools

List of available topics: `ros2 topic list`

Publish on a topic: `ros2 topic pub <topic name> <msg type> <content>`

Listen to topic updates: `ros2 topic echo <topic name>`

Get information about the topic: `ros2 topic info <topic name>`

more on ROS topics [here](#)

# ROS Messages

Messages describe the data structures

that record the information to be exchanged among the nodes

Composed of fields that have a name and a type

A type may be either another message or a primitive type

examples of primitive types: bool, int32, float64

Checking the message structure: `ros2 interface show <message type>`

## Examples

`std_msgs/msg/Float64`  
float64 data

`geometry_msgs/Vector3`  
float64 x  
float64 y  
float64 z

`geometry_msgs/msg/Twist`  
Vector3 linear  
Vector3 angular

Go to exercises 1.1.1 and 1.1.2

more on ROS messages [here](#)

# The Turtlesim simulation

## Subscribed topics

```
turtleX/cmd_vel [geometry_msgs/msg/Twist]
```

Records the intended linear and angular velocity of the robot

The robot controller reads the topic and adjusts its velocity accordingly

keeping the velocity for 1 second

`geometry_msgs/msg/Twist`

|                              |                                                                                              |
|------------------------------|----------------------------------------------------------------------------------------------|
| <code>Vector3 linear</code>  | <code>linear.x</code> : controls the forward ( $x > 0$ ) and backward ( $x < 0$ ) movement   |
| <code>float64 x</code>       | <code>linear.y</code> : controls the upward ( $y > 0$ ) and downward ( $y < 0$ ) movement    |
| <code>float64 y</code>       | <code>linear.z</code> : no effect in turtlesim                                               |
| <code>float64 z</code>       | <code>angular.x</code> : no effect in turtlesim                                              |
| <code>Vector3 angular</code> | <code>angular.y</code> : no effect in turtlesim                                              |
| <code>float64 x</code>       | <code>angular.z</code> : controls the clockwise ( $z < 0$ ) and counterclockwise ( $z > 0$ ) |
| <code>float64 y</code>       | movement around the robot's axis                                                             |
| <code>float64 z</code>       |                                                                                              |

# The Turtlesim simulation

## Published topics

`/turtle1/pose [turtlesim/msg/Pose]`

Records the robot's position, angle, and linear and angular velocity

the robot updates this topic when these values change.

`turtlesim/msg/Pose`

|                                       |                                                                    |
|---------------------------------------|--------------------------------------------------------------------|
| <code>float32 x</code>                | <code>x</code> : position of the robot in the x axis               |
| <code>float32 y</code>                | <code>y</code> : position of the robot in the y axis               |
| <code>float32 theta</code>            | <code>theta</code> : angle of the robot with respect to the x axis |
| <code>float32 linear_velocity</code>  | <code>linear_velocity</code> : linear velocity of the robot        |
| <code>float32 angular_velocity</code> | <code>angular_velocity</code> : linear velocity of the robot       |

Go to exercise 1.1.3

# ROS Services

Services are functions provided by the nodes

to be requested by other nodes following a client/server pattern with blocking communication

May return some information as result of the execution

Services have a *type* that defines the *message type* of both the request and response

## useful command line topic tools

List of available services: `ros2 service list`

Call a service: `ros2 service call <service name> <service args>`

Get the service type: `ros2 service type <service name>`

Go to exercises 1.2.1 and 1.2.2

more on ROS services [here](#)

# The Turtlesim simulation

## Provided Services

`/turtle1/teleport_absolute [turtlesim/srv/TeleportAbsolute]`

Moves the robot's to a given point  $(x, y)$  in the environment

and rotates the robot if needed

`turtlesim/srv/TeleportAbsolute`

`float32 x`

`float32 y`

`float32 theta`

x: target position in the x axis

y: target position in the y axis

theta: target angle with respect to the x axis

in radians  $\alpha_{rad} = \frac{\alpha^{\circ} \times \pi}{180}$

Go to exercise 1.2.3

# The Turtlesim simulation

## Provided Services

`/turtle1/teleport_Relative [turtlesim/srv/TeleportRelative]`

Moves the robot's along a given distance forward/backward

and rotates the robot if needed

`turtlesim/srv/TeleportRelative`

`float32 linear`  
`float32 angular`

linear: distance to move the robot forward/backward

angular: angle to rotate the robot  
in radians  $\alpha_{rad} = \frac{\alpha^{\circ} \times \pi}{180}$

Go to exercices 1.2.4 and 1.2.5

— Part II —  
BDI Agents  
Jason

# What is an agent?

*“... anything ... perceiving its environment through sensors and acting upon that environment through actuators”* [Russel and Norvig, 1995]

*“... a computer system that is situated in some environment, and that is capable of autonomous action in this environment in order to meet its delegated objectives..”* [Wooldridge, 2009]

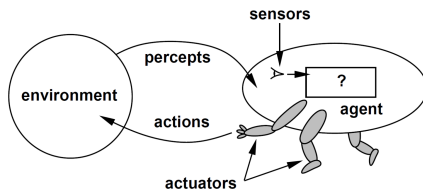


Image from [Russel and Norvig, 1995]

# The BDI Architecture

## **B**eliefs, **D**esires, and **I**ntentions

### **Beliefs**

Information that the agent possesses about the world in which it operates

### **Desires**

States of the world that the agent would like to achieve (*options*)

### **Intentions**

States of the world that the agent has decided to act upon to achieve

# The BDI Architecture

## Example: autonomous vehicle

Beliefs: current position, current speed, available energy

Desires: reach the destination quickly, maintain energy above a certain threshold

Intentions: if energy falls below the threshold: acquire energy

# The BDI Architecture

How does an agent go from its BDI to actions?

## Practical Reasoning

Deciding among options (based on its knowledge)

which can even be conflicting (e.g., reach the destination quickly vs. maintain energy)

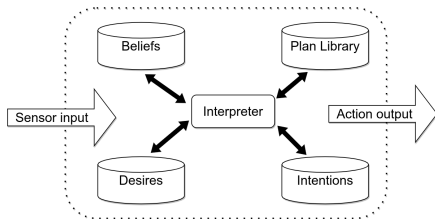
## Practical Reasoning Involves Two Processes

1. *Deliberation*: deciding which intentions to adopt
2. *Means-End Reasoning*: deciding the sequence of actions to follow to satisfy the adopted intentions

# The BDI Architecture

How does a BDI computer program behave?

1. Perceives the world and updates beliefs
2. Evaluates its desires and decides which intention(s) to achieve
3. Through means-end reasoning, defines a plan to satisfy the intention
4. Executes the plan



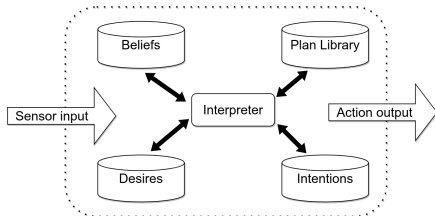
Adapted from [Bordini et al., 2007]

# How to Implement BDI?

How to implement programs with BDI behavior?

It is necessary to implement:

1. perception mechanisms
2. representation of beliefs, desires, plans
3. belief revision
4. generation of intentions
5. deliberation
6. means-end reasoning
7. communication mechanisms
8. plan execution mechanisms
9. etc, etc, etc...
10. agent behavior



Adapted from [Bordini et al., 2007]

Agent-oriented programming: languages and interpreters implement items 1 to 9 above. The programmer focuses on item 10.

# The Jason Language

A language for programming BDI agents

<https://jason-lang.github.io>

In this course, *Jason* is used with the [JaCaMo framework](#) [Boissier et al., 2020]



## Agent Programming with high-level BDI constructs

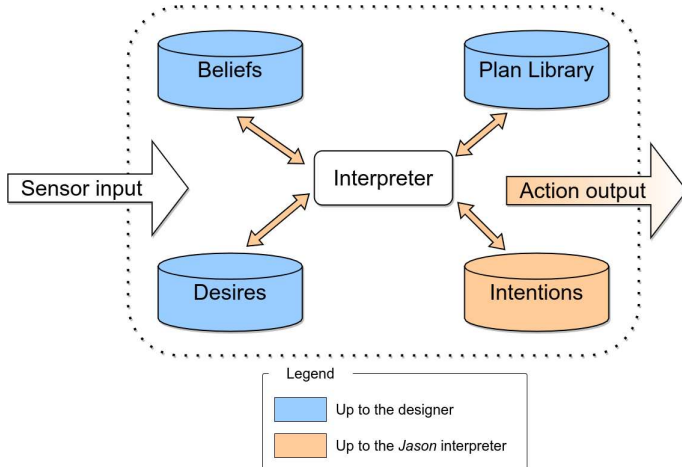
in Jason, we program beliefs, desires, and intentions

instead of programming variables, functions, procedures, methods, objects, ...

## Agent Execution in the form of a reasoning cycle

a Jason agent remains in continuous execution, observing the world, updating its beliefs  
evaluating the desires that can be satisfied, selecting plans to execute them,  
putting the plans into execution, and restarting the cycle

# The Jason Language



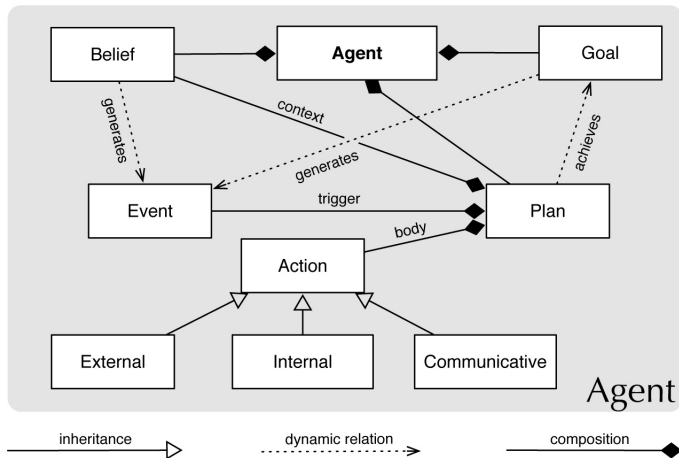
# Agent Oriented Programming with *Jason*<sub>[Bordini et al., 2007]</sub>

MAS development platform based on an extended version of AgentSpeak  
[Rao, 1996]

Agents are implemented with high level constructs of the BDI model  
[Bratman et al., 1988]

In this course, *Jason* is used with the JaCaMo framework [Boissier et al., 2020]

# Agents



[Boissier et al., 2020]

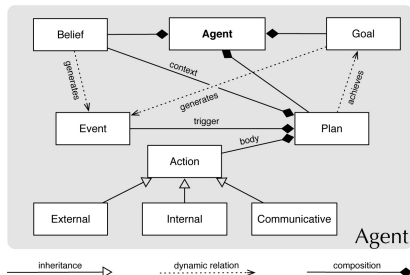
# Agents

## Essential Jason constructs

**Beliefs:** information available to the agent about the environment, other agents, itself...

**Goals:** states of affairs the agent wants to bring about

**Plans:** recipes for action (agent's know how)



[Boissier et al., 2020]

# Hello World - The first JaCaMo program

```
language(english). initial belief
!greet. //initial desire (or goal)

//plans to satisfy the goal greet
+!greet : language(english)
    <- .print("hello world.").

+!greet : language(french)
    <- .print("bonjour.").
```

# Hello World - The first JaCaMo application - version 1

## JaCaMo app specification

```
mas agents_intro {  
  agent bob: personal_assistant.asl  
}
```

## agent specification

```
language(english). initial belief  
!greet. //initial desire (or goal)  
  
//plans to satisfy the goal  
+!greet : language(english)  
  <- .print("hello world.").  
  
+!greet : language(french)  
  <- .print("bonjour.").
```

# Hello World - The first JaCaMo application - version 2

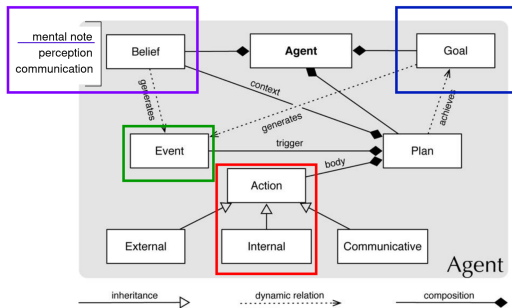
## JaCaMo app specification

```
mas agents_intro {  
  agent bob: personal_assistant.asl{  
    goals: greet  
    beliefs: language(english)  
  }  
}
```

## agent specification

```
//plans to satisfy the goal  
+!greet : language(english)  
  <- .print("hello world.").  
  
+!greet : language(french)  
  <- .print("bonjour.").
```

# Hello World - The first JaCaMo program



adapted from [Boissier et al., 2020]

## JaCaMo app specification

```
mas agents.intro {  
  agent bob: personal_assistant.asl {  
    goals: greet  
    beliefs: language(english)  
  }  
}
```

## agent specification

```
!greet. //initial desire (or goal)  
  
//plans to satisfy the goal  
+!greet : language(english)  
  <- .print("hello world").  
  
+!greet : language(french)  
  <- .print("bonjour").
```

Go to exercises 2.1.1 to 2.3.3

# Main agent constructs – Beliefs

## Syntax

Beliefs are represented by annotated literals of first order logic

*functor*(*term*<sub>1</sub>,  $\dots$ , *term*<sub>*n*</sub>)[*annot*<sub>1</sub>,  $\dots$ , *annot*<sub>*n*</sub>]

## Example

```
language(english)[source(self)] //added by the agent itself  
temperature(20)[source(percept)] //added by perception  
friend(bob,alice)[source(bob)] //added by communication from bob
```

# Where do the beliefs come from? I

Coded as agent's initial beliefs either in the .asl or in the .jcm file

```
language(english). //initial belief
!greet. //initial goal

//plan to satisfy the goal
+!greet : language(english)
  <- print('Hello world').
```

Added as part of plans

```
language(english). //initial belief
!greet. //initial goal

//plan to satisfy the desire
+!greet : language(english)
  <- print('Hello world').
  -last_greeting_date(friday); //remove belief
  +last_greeting_date(saturday); //add belief
  +last_greeting_date(sunday). //update belief
```

# Where do the beliefs come from? II

## Beliefs added by communication

- Have the `source(ag)` annotation

where `ag` is the agent that provides the information

- Dynamics:

- when the agent receives a *tell* message, its content becomes a new belief

```
.send(tom,tell,lier(alice)); //sent by bob  
//adds lier(alice)[source(bob)] in tom's bb
```

- when the agent receives an *untell* message, the corresponding belief is removed from the receiver's belief base

```
.send(tom,untell,lier(alice)); //sent by bob  
//removes lier(alice)[source(bob)] in tom's bb
```

Go to exercise 2.3.4

# Where do the beliefs come from? III

From the agent's perception of the environment

include the annotation `source(percept)`  
are automatically updated as perceptions change

```
language(english).  
  
!greet.  
  
+!greet : language(english)  
          temperature(X) [source(percept)] & X <= 10  
  <- .print("hello world.");  
      .print("It is so cool").
```

Go to Exercise 2.3.5

# Main agent constructs – Goals

**Goals** correspond to the notion of *desire* from BDI.

Represent the states of the world the agent wants to bring about

## Syntax

`! < goal_identifier > (< argument1, ..., argumentn >)`

(arguments are optional)

Types of goals:

- declarative goals: goals about states (e.g. `!be_at(wesaac)`)
- procedural goals: goals about (sequence of) actions (e.g. `!go_to(wesaac)`)

Declarative and procedural goals are *achievement goals*, which are goals to do.

Jason considers also *test goals*, which are goals to know. Test goals are not the focus in this presentation.

# Where do the goals come from? I

## Agent's initial goal either in the .asl or in the .jcm file

```
language(english). //belief
!greet. //initial goal

//plan to satisfy the goal
+!greet : language(english)
  <- print("Hello world").
```

## Part of plans

```
language(english). //belief
!greet. //initial goal

//plan to satisfy the desire
+!greet : language(english)
  <- print("Hello world.");
    //add test [sub]goal
    ?raining(now);
    //add achievement [sub]goal
    !check_temperature.
```

# Where do the goals come from? II

## Goals added by communication

- when the agent receives an *achieve* message, its content becomes an achievement goal annotated with the sender of the message

```
.send(tom,achieve,write(book));//sent by bob  
//adds new goal write(book)[source(bob)] for Tom
```

- when the agent receives an *unachieve* message, the goal corresponding to the message's content is removed

```
.send(tom,unachieve,write(book));//sent by bob  
//removes the goal write(book)[source(bob)] for Tom
```

- when the agent receives an *askOne* or *askAll* message, the content is a new test goal annotated with the sender of the message

```
.send(tom,askOne,published(P),Answer);//sent by bob  
//adds new goal ?published(P)[source(bob)] for Tom  
//the response of Tom unifies with Answer
```

# Plans

```
triggering_event : context <-- body.
```

triggering\_event: event the plan is meant to handle

context: circumstances in which the plan is selected

body: course of action to be followed when the plan is selected

## Triggering events

- +b (belief addition)
- b (belief deletion)
- +!g (achievement goal addition)
- !g (achievement goal deletion)
- +?g (test goal addition)
- ?g (test goal deletion)

# Plans – context

Context is typically formed of conjunctions of beliefs that the agent is required to have at the time of choosing the plan

```
+!greet : language(english) & temperature(X) & X>=30  
<- . . .
```

## Boolean operators

|              |              |
|--------------|--------------|
| &            | (and)        |
|              | (or)         |
| not          | (not)        |
| =            | (not)        |
| >, >=, <, <= | (relational) |
| ==           | (equals)     |
| \ ==         | (different)  |

## Arithmetic operators

|            |                  |
|------------|------------------|
| +          | (sum)            |
| -          | (subtraction)    |
| *          | (multiply)       |
| /          | (divide)         |
| <b>div</b> | divide – integer |
| <b>mod</b> | remainder        |
| **         | power            |

# Plans - body

What may the plan body include?

```
+rain : time_to_leave(T) & clock.now(H) & H>=T
<- !g1; //new subgoal
    !!g2; //new goal
    ?b(X); //new test goal
    +b1(T-H); //add mental note
    -b2(T+H); //remove mental note
    -+b3(T*H); //update mental note
    close(door); //external action
    .print(`Go!'). internal action
```

# Plans vs. actions of agents

Plans may include **actions** the agents are expected to execute.

But...

Where are these actions implemented?

There are two kinds of actions:

- Internal actions
- External actions

# External actions

Executed on elements external to the agent's implementation i.e.  
*environment*

In JaCaMo, the environment is based on CArTAgO (details [here](#) and [here](#))

# Internal actions

- are not necessarily performed upon an external device
- are part of the agent implementation
- are executed as part of the agent reasoning cycle
- are available to the agent in any environment

*Internal actions* are implemented in Java

- New *internal actions* can be implemented and added to the agent
- Jason provides a default set of internal actions available to every Jason agent
- More info [here](#)

Example (download [here](#))

```
+!start <-  
  .print("hello world.");  
  .random(R); .print(R);  
  .date(Y,M,D);  
  .print(Y,"-",M,"-",D);  
  .time(H,MN,S);  
  .print(H,"-",MN,"-",S).
```

# Part III

## Introduction to Jason-ROS

# Autonomous robots with BDI agents and ROS

## Assumptions:

- BDI agents are a suitable approach for developing systems with features such as proactivity, autonomy, deliberative skills, and social abilities
- ROS is a suitable tool for robot programming  
but does not provide features such as proactivity, autonomy, deliberative skills, and social abilities
- Conciliating BDI agent programming with ROS to have robots with these features
  - integrate the BDI model with the ROS programming model
  - integrate AOP tools with the ROS platform

# Autonomous robots with Jason and ROS

## **Assumptions:**

Jason is suitable for BDI agent programming

the behaviour of a robot may be fully specified in Jason

Programming Jason-ROS robots requires to connect

- perceptions of the agent with the robot sensors  
whose values are recorded in ROS topics
- actions of the agent with the physical effectors  
controlled by both ROS topics and actions

Jason-ROS extends the default Jason agents:

- perceptions may be obtained from physical sensors  
whose values are recorded in ROS topics
- actions may be realized by physical actuators  
controlled by ROS topics and services

Jason-ROS is available [here](#).

It is part of a more comprehensive work to combine Jason agents with physical devices available [here](#).

# Jason-ROS

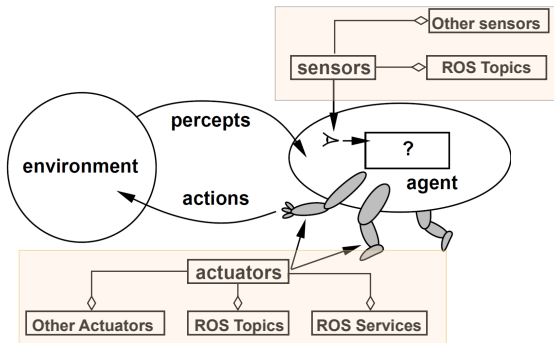
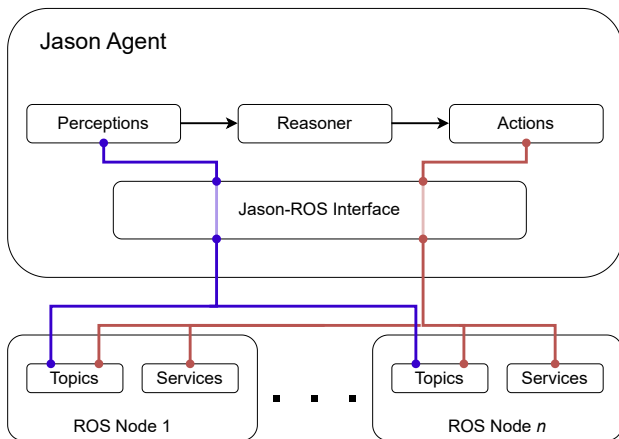
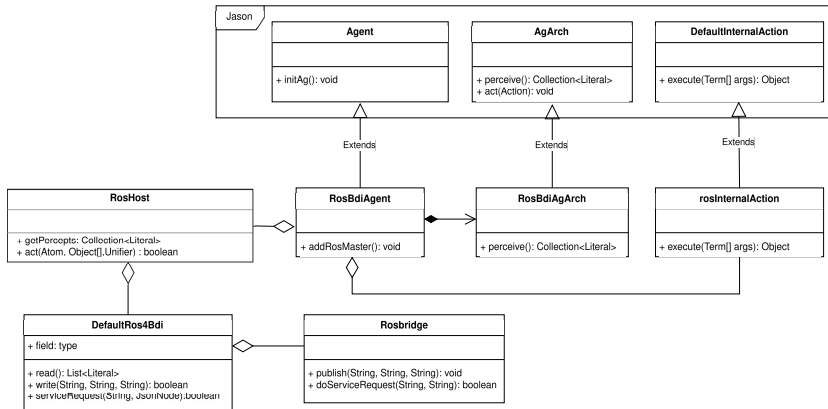


Image adapted from [Russel and Norvig, 1995]

# Jason-ROS



# Jason-ROS



## Set up of a Jason-ROS agent in JaCaMo

```
mas jason_ros {  
  agent robot1: ros_agent.asl {  
    ag-class: embedded.mas.bridges.jacamo.ros.RosBdiAgent  
    ag-arch: embedded.mas.bridges.jacamo.ros.RosBdiAgArch  
  }  
}
```

**ag-class**: defines the Java class that implements the essential functions and data structures for running an agent

### **RosBdiAgent**:

- extends the default Agent class of Jason
- provides means for
  - including topics among the perception sources of the agent
  - linking the action repertory to service calls and topic writings

## Set up of a Jason-ROS agent in JaCaMo

```
mas jason_ros {  
  agent robot1: ros_agent.asl {  
    ag-class: embedded.mas.bridges.jacamo.ros.RosBdiAgent  
    ag-arch : embedded.mas.bridges.jacamo.ros.RosBdiAgArch  
  }  
}
```

**ag-arch** : defines the Java class that implements the essential functions and data structures for the agent to interact with the external world

in the perceive and act phases of the reasoning cycle

**RosBdiAgArch** :

- extends the [default agent architecture of Jason](#)
- includes the reading of topics in the perceiving process

# Programming Jason-ROS Robots

Connecting the agents' abstractions to ROS resources

The cognitive portion of the agent is programmed in .asl files

The connection between cognitive and physical portion (controlled by ROS) is specified in a .yaml configuration file

- each agent has its own .yaml file

- the .yaml file has the same name as the agent name (which may differ from the .asl file name)

- the same .asl may lead to agents with different connections Jason-ROS

# Programming Jason-ROS Robots

Connecting the agents' abstractions to ROS resources

A Jason-ROS agent is composed of a set of *devices*

A *device* is a physical portion of the agent,  
composed of a set of sensors and actuators (controlled by ROS, in this case)

Jason-ROS configuration file

```
- device_id: device_1
  ...
- device_id: device_2
  ...
- device_id: device_n
```

# Programming Jason-ROS Robots

## Connecting the agents' abstractions to ROS resources

**device\_id:** identifies a connection with an external portion of the agent that, in this case, is a ROS host

### Jason-ROS configuration file

```
- device_id: sample_roshost
  className: RosMaster
  microcontroller:
    id: ros1
    connectionString: ws://localhost:9090
    className: DefaultRos4Bdi
  perceptionTopics:
    ...
  actions:
    ...
```

each ROS device is defined by a set of parameters

- a class that implements the interface Jason-ROS
- a microcontroller, that abstracts the connection parameters of this interface
- a configuration of perceptions acquired from ROS
- a configuration of actions realized by ROS

an agent may be connected with many devices (including non-ROS)

the value (e.g. `sample_roshost`) is defined by the developer

# Programming Jason-ROS Robots

Connecting the agents' abstractions to ROS resources

**device\_id:** identifies a connection with an external portion of the agent  
that, in this case, is a ROS host

## Jason-ROS configuration file

```
- device_id: sample_roshost
  className: RosMaster
  microcontroller:
    id: ros1
    connectionString: ws://localhost:9090
    className: DefaultRos4Bdi
  perceptionTopics:
    ...
  actions:
    ...
```

each ROS device is defined by a set of parameters

- a class that implements the interface Jason-ROS
- a microcontroller, that abstracts the connection parameters of this interface
- a configuration of perceptions acquired from ROS
- a configuration of actions realized by ROS

an agent may be connected with many devices (including non-ROS)

the value (e.g. `sample_roshost`) is defined by the developer

# Programming Jason-ROS Robots

Connecting the agents' abstractions to ROS resources

**device\_id:** identifies a connection with an external portion of the agent that, in this case, is a ROS host

## Jason-ROS configuration file

```
- device_id: sample_roshost
  className: RosMaster
  microcontroller:
    id: rosl
    connectionString: ws://localhost:9090
    className: DefaultRos4Bdi
  perceptionTopics:
    ...
  actions:
    ...
```

each ROS device is defined by a set of parameters

- a class that implements the interface Jason-ROS
- a microcontroller, that abstracts the connection parameters of this interface
- a configuration of perceptions acquired from ROS
- a configuration of actions realized by ROS

an agent may be connected with many devices (including non-ROS)

the value (e.g. `sample_roshost`) is defined by the developer

# Programming Jason-ROS Robots

## Connecting the agents' abstractions to ROS resources

**device\_id:** identifies a connection with an external portion of the agent that, in this case, is a ROS host

### Jason-ROS configuration file

```
- device_id: sample_roshost
  className: RosMaster
  microcontroller:
    id: ros1
    connectionString: ws://localhost:9090
    className: DefaultRos4Bdi
    perceptionTopics :
      ...
  actions:
    ...
```

each ROS device is defined by a set of parameters

- a class that implements the interface Jason-ROS
- a microcontroller, that abstracts the connection parameters of this interface
- a configuration of perceptions acquired from ROS
- a configuration of actions realized by ROS

an agent may be connected with many devices (including non-ROS)

the value (e.g. `sample_roshost`) is defined by the developer

# Programming Jason-ROS Robots

Connecting the agents' abstractions to ROS resources

**device\_id:** identifies a connection with an external portion of the agent that, in this case, is a ROS host

## Jason-ROS configuration file

```
- device_id: sample_roshost
  className: RosMaster
  microcontroller:
    id: ros1
    connectionString: ws://localhost:9090
    className: DefaultRos4Bdi
  perceptionTopics:
    ...
  actions :
```

each ROS device is defined by a set of parameters

- a class that implements the interface Jason-ROS
- a microcontroller, that abstracts the connection parameters of this interface
- a configuration of perceptions acquired from ROS
- a configuration of actions realized by ROS

an agent may be connected with many devices (including non-ROS)

the value (e.g. `sample_roshost`) is defined by the developer

# Programming Jason-ROS Robots

Connecting the agents' abstractions to ROS resources

**perceptionTopics** : connects the agents' perception system with the physical/simulated sensors which produce perceptions that eventually become beliefs

## Jason-ROS configuration file

```
- device_id: sample_roshost
...
perceptionTopics :
  - topicName: /turtle1/energy
    topicType: std_msgs/Int32
    beliefName: batteryLevel
  - topicName: /parameter_events
    topicType: rcl_interfaces/ParameterEvent
    beliefName: known_events
    ignoreValues:
      - changed_parameters
      - deleted_parameters
actions:
...
```

# Programming Jason-ROS Robots

Connecting the agents' abstractions to ROS resources

**perceptionTopics:** connects the agents' perception system with the physical/simulated sensors  
which, produce perceptions and beliefs that eventually produce beliefs

## Jason-ROS configuration file

```
- device_id: sample_roshost
  ...
- perceptionTopics :
  - topicName: /turtle1/energy
    topicType: std_msgs/Int32
    beliefName: batteryLevel
  ...
```

- each item specifies a connection between a topic and a perception/belief
- the keys **topicName** and **topicType** have an intuitive meaning
- **beliefName**: name (or *functor*) of the produced perception

# Programming Jason-ROS Robots

## Connecting the agents' abstractions to ROS resources

By default, if the topic value has a single field, the produced perception has the form  $b(v)$ , where (i)  $b$  is the **beliefName** and  $v$  is field value.

### Jason-ROS configuration file

```
- device_id: sample_roshost
  ...
- perceptionTopics :
  - topicName: /turtle1/energy
    topicType: std_msgs/Int32
    beliefName: batteryLevel
  ...
```

e.g.: if the value recorded in the topic `/turtle1/energy` is 100, then the agent has the belief *batteryLevel(100)*

# Programming Jason-ROS Robots

## Connecting the agents' abstractions to ROS resources

By default, if the topic value has  $n$  fields (s.t.  $n > 1$ ), the produced perception has the form  $b(f_1(v_{f_1}), \dots, f_n(v_{f_n}))$ , where (i)  $b$  is the **beliefName**, (ii)  $(f_i)$  is the name of the  $i^{th}$  field, and (iii)  $v_{f_i}$  is the value of the  $i^{th}$  field.

### Jason-ROS configuration file

```
- device_id: sample_roshost
  ...
- perceptionTopics :
  - topicName: /turtle1/energy
    topicType: std_msgs/Int32
    beliefName: energyStatus
  ...
```

e.g.: if the topic `/turtle1/energy` has the fields `batteryLevel` and `remainingTime`, then the agent has the belief `energyStatus(batteryLevel(100),remainingTime(60))`

# Programming Jason-ROS Robots

## Connecting the agents' abstractions to ROS resources

In multi-field topics, the key `ignoreValues` specifies topic fields that do not belong to the belief.

### Jason-ROS configuration file

```
- device_id: sample_roshost
  ...
- perceptionTopics :
  - topicName: /parameter_events
    topicType: rcl_interfaces/ParameterEvent
    beliefName: known_events
    ignoreValues :
      - changed_parameters
      - deleted_parameters
    ...
```

e.g.: the topic `/parameter_events` has 5 fields:  $f_1$ =stamp,  $f_2$ =node,  $f_3$ =new\_parameters,  $f_4$ =changed\_parameters,  $f_5$ =deleted\_parameters

- if the key `ignoreValues` is omitted, the corresponding belief is  $known\_events(f_1, f_2, f_3, f_4, f_5)$ .
- otherwise, the corresponding belief is  $known\_events(f_1, f_2, f_3)$ .

Customizations in ROS-based beliefs are discussed later in this course

# Programming Jason-ROS Robots

Connecting the agents' abstractions to ROS resources

Nested fields to be ignored are chained with a “.”

## Jason-ROS configuration file

```
- device_id: sample_roshost
  ...
- perceptionTopics :
  - topicName: turtle1/cmd_vel
    topicType: geometry_msgs/Twist
    beliefName: velocity
    ignoreValues :
      - linear.z
      - angular
  ...
```

e.g.: the message `geometry_msgs/Twist` has the following structure:

```
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z
```

- ignoring the field `angular` implies to ignore `angular.x`, `angular.y`, and `angular.z`
- to ignore the field `z` nested in `linear`, the ignore value is set to `linear.z`

# Programming Jason-ROS Robots

Connecting the agents' abstractions to ROS resources

**actions** : connects the agents' actions with the physical/simulated effectors  
controlled by ROS topics and services

## Jason-ROS configuration file

```
- device_id: sample_roshost
  ...
  actions :
    topicWritingActions :
      ...
    serviceRequestActions :
```

- **topicWritingActions** :  
actions realized through topic writings
- **serviceRequestActions** :  
actions realized through service requests

# Programming Jason-ROS Robots

## Configuring service-based actions

### Jason-ROS configuration file

```
- device_id: sample_roshost
  ...
  actions:
    ...
    serviceRequestActions:
      - actionName: recharge_battery
        serviceName: /turtle1/do_recharge
        params:
          - time
```

**actionName**: name of the action in the agent's repertory

in this case, the agent's action repertory includes an action recharge\_battery

**actionName** is not necessarily related to hardware specifics

(it is more related to the agent's application domain)

the hardware can change without changing the agents' action repertory

# Programming Jason-ROS Robots

## Configuring service-based actions

### Jason-ROS configuration file

```
- device_id: sample_roshost
  ...
  actions:
    ...
    serviceRequestActions:
      - actionName: recharge_battery
        serviceName: /turtle1/do_recharge
        params:
          - time
          - charging_current
```

**serviceName**: name of the ROS service that actually realizes that action  
in this case, the agent's action `recharge_battery`  
is realized through the service `/turtle1/do_recharge`  
(which is supposed to control some actuator).

# Programming Jason-ROS Robots

## Configuring service-based actions

### Jason-ROS configuration file

```
- device_id: sample_roshost
  ...
  actions:
    ...
    serviceRequestActions:
      - actionName: recharge_battery
        serviceName: /turtle1/do_recharge
        params :
          - time
          - charging_current
```

**params** : list of parameters required by the service call

the parameters list must be the same as the service specification

this specification can be checked with the command `ros2 interface show <service type>`

the service type can be checked with the command `ros2 service type <service name>`

to get the service parameters from its name, use

```
ros2 service type <service name> | xargs ros2 interface show
```

# Programming Jason-ROS Robots

## Executing service-based actions

ROS-based actions become **internal actions**

Assumption: these actions belong to the agent implementation  
instead of being provided by elements from the environment

### Jason-ROS configuration file

```
- device_id: sample_roshost
...
actions:
...
  serviceRequestActions:
    - actionName: recharge_battery
      serviceName: /turtle1/do_recharge
      params:
        - time
        - charging_current
```

### Agent program

```
+triggering_event : context
<- ...
  recharge_battery ( 30 , 600 );
...
```

The internal action has the same parameters as the ROS service (in the same order)

The ROS-based internal actions are stored in `src/java/jason/stdlib`

# Programming Jason-ROS Robots

## Configuring topic-based actions

### Jason-ROS configuration file

```
- device_id: sample_roshost
...
actions:
...
topicWritingActions:
- actionName: move_robot
  topicName: /turtle1/cmd_vel
  topicType: geometry_msgs/Twist
  params:
    - linear:
      - x
      - y
      - z
    - angular:
      - x
      - y
      - z
```

**actionName**: name of the action in the agent's repertory  
in this case, the agent's action repertory includes an action `move_robot`  
**actionName** is not necessarily related to hardware specifics  
(it is more related to the agent's application domain)  
the hardware can change without changing the agents' action repertory

**topicName** and **topicType** have intuitive meaning

# Programming Jason-ROS Robots

## Configuring topic-based actions

### Jason-ROS configuration file

```
- device_id: sample_roshost
...
actions:
...
  topicWritingActions:
    - actionName: move_robot
      topicName: /turtle1/cmd_vel
      topicType: geometry_msgs/Twist
      params :
        - linear:
            - x
            - y
            - z
        - angular:
            - x
            - y
            - z
```

**params** : list of fields recorded in the topic

the parameters list must be the same as the topic type specification

this specification can be checked with the command `ros2 interface show <topic type>`

the topic type can be checked with the command `ros2 service type <topic name>`

to get the service parameters from its name, use

`ros2 topic type <topic name> | xargs ros2 interface show`

# Programming Jason-ROS Robots

## Executing topic-based actions

### Jason-ROS configuration file

```
- device_id: sample_roshost
  ...
  actions:
    ...
    topicWritingActions:
      - actionName: move_robot
        topicName: /turtle1/cmd_vel
        topicType: geometry_msgs/Twist
        params:
          - linear:
              - x
              - y
              - z
          - angular:
              - x
              - y
              - z
```

### Agent program

```
+triggering_event : context
  <- ...
    .move_robot ( [1,1,0] , [0,0,1] );
  ...
```

The internal action has the same parameters as the ROS topic type (in the same order)

The ROS-based internal actions are stored in `src/java/jason/stdlib`

An application example of topic-based actions is available [here](#)

# Advanced topics on Jason-ROS

Customization of actions

Multi-agent examples

# References I



Multi-Agent Oriented Programming: Programming Multi-Agent Systems Using JaCaMo. 1 ed. The MIT Press, 2020.



Rafael H. Bordini, Jomi F. Hübner, Michael Wooldridge

Programming Multi-Agent Systems in AgentSpeak using Jason.. 1 ed. Willey, 2007.



Michael E. Bratman and David J. Israel and Martha E. Pollack

Plans and resource-bounded practical reasoning. *Computational Intelligence*, 1988.



Jomi Fred Hübner and Jaime Simão Sichman and Olivier Boissier

Developing organised multiagent systems using the MOISE<sup>+</sup> model: programming issues at the system and agent levels. Int. J. Agent Oriented Softw. Eng., 2007.



Anand S. Rao

AgentSpeak(L): BDI agents speak out in a logical computable language. *Agents Breaking Away* Walter Van de Velde and John W. Perram (Eds.). Springer Berlin Heidelberg, 1996.



Stuart Russel and Peter Norvig.

Artificial Intelligence: A Modern Approach. 1 ed. Prentice Hall, 1995.

# References II



Alessandro Ricci and Michele Piunti and Virko Viroli and Andrea Omicini  
Environment Programming in CArtaGO. Multi-Agent Programming,  
Languages, Tools and Applications, 2009.



SEARLE, J. The Construction of Social Reality. [S.I.]: Free Press, 1995.



SEARLE, J. Making the Social World:The Structure of Human Civilization.  
[S.I.]: Oxford University Press, 2009.



Michael Wooldridge

An Introduction to MultiAgent Systems.. 2 ed. Willey, 2009.